Spécification du moteur de mapping FtM

Oct 22, 2026 · @Ahmed Sairi

1. Objectif et périmètre

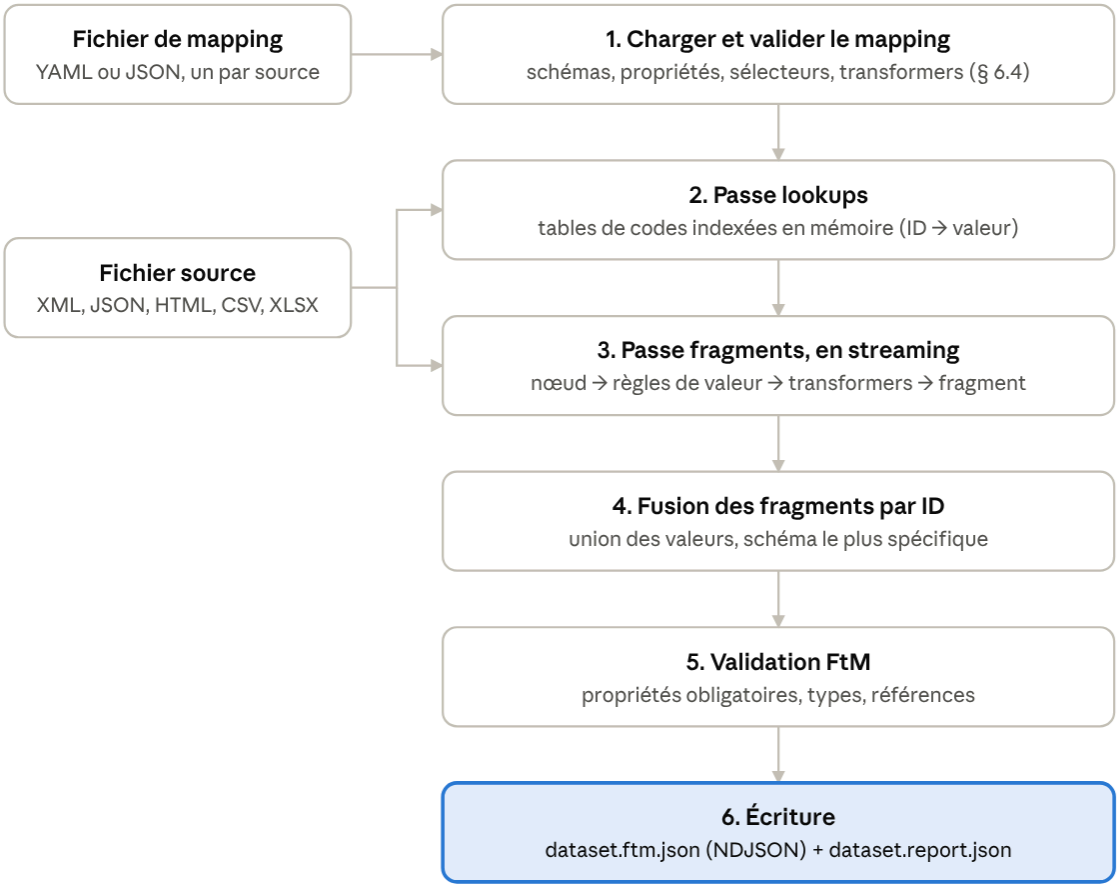
Un seul moteur Java transforme n'importe quelle source (XML, JSON, HTML, CSV, XLSX) en entités FollowTheMoney au format NDJSON. Toute la logique propre à une source tient dans **un fichier de mapping** (YAML ou JSON, même structure). Ajouter une source revient à écrire un mapping, sans recompiler.

Dans le périmètre	Hors périmètre
Lecture d'un fichier local déjà téléchargé	Téléchargement, pagination, authentification (couche *extract* en amont)
Découpage en enregistrements, extraction, normalisation	PDF et HTML non structuré (parser Java dédié via `SourceParser`)
Tables de codes (`lookups`), liens entre entités (`ref`), fusion des fragments	Déduplication entre sources (étape de *matching* en aval)
Validation contre le modèle FtM (`tech.followthemoney:followthemoney`)	Indexation (yente, Elasticsearch)

Sortie : un fichier `<dataset>.ftm.json`, une entité JSON par ligne, produite par `ValueEntity.toValueJson()`, plus un rapport d'exécution.

2. Pipeline d'exécution

Deux passes sur la source, puis fusion, validation et écriture



pipeline du moteur · 6 étapes, 2 passes sur la source

La source est lue deux fois : d'abord pour indexer les petites tables de codes, puis en streaming pour produire les fragments. Rien n'est écrit tant que la fusion et la validation ne sont pas terminées.

3. Format du fichier de configuration

Un mapping = un fichier par source, nommé `<dataset>.mapping.yaml` ou `.mapping.json`. Le moteur le lit dans un modèle Java unique (`MappingConfig`), quel que soit le format.

3.1 Clés racine

Clé	Type	Obligatoire	Rôle
`version`	entier	oui	Version du format de mapping (`1`). Permet de faire évoluer la spec.
`dataset`	chaîne `[a-z0-9_-]+`	oui	Nom du dataset FtM. Préfixe de tous les IDs et valeur de `datasets` .
`format`	`json` | `xml` | `html` | `csv` | `xlsx`	oui	Choisit le `RecordReader` .
`reader`	objet	non	Options du lecteur : `encoding` , `namespaces` (XML), `delimiter` / `header` (CSV), `sheet` (XLSX).
`defaults`	objet	non	Propriétés ajoutées à chaque entité (ex. `sourceUrl` , `publisher`).
`lookups`	map nom → lookup	non	Tables de codes chargées avant les fragments (§ 3.2).
`fragments`	liste de fragments	oui	Règles de création d'entités (§ 3.3).

3.2 Lookup

Clé	Type	Obligatoire	Rôle
`path`	sélecteur	oui	Où sont les lignes de la table (ex. `//NamePartType`).
`key`	sélecteur	oui	Valeur qui sert de clé (ex. `@ID`).
`value`	sélecteur	non	Valeur retournée. Absente : l'élément entier est gardé, lisible ensuite avec `field` .
`map`	map	non	Table statique écrite dans le YAML, sans `path` (ex. `{ M: male, F: female }`).

3.3 Fragment

Clé	Type	Obligatoire	Rôle
`name`	chaîne	oui	Nom unique de la règle, cité dans les logs.

Clé	Type	Obligatoire	Rôle
`node`	sélecteur	oui	Racine d'itération : 1 nœud trouvé = 1 fragment (ex. `$.notices[*]`, `//Profile`, `table.deputes tr`).
`when`	condition	non	Filtre : le nœud n'est traité que si la condition est vraie (§ 4).
`schema`	chaîne ou règle de valeur + `map`	oui	Schéma FtM fixe (`Person`) ou calculé (§ 4.6).
`id`	`{ ns, keys }`	oui	Construction de l'ID déterministe (§ 6).
`properties`	map propriété FtM → règle de valeur	oui	Valeurs à ajouter (§ 4).

3.4 Sélecteurs par format

Le même mot `path` change de langage selon `format`. Tous les sélecteurs d'un fragment sont relatifs à son `node`.

format	Langage	Exemple	Lib Java
json	JSONPath	`$.nationalities[*]`	Jayway `json-path`
xml	XPath 1.0	`NamePart[@TypeID='1521']`	`javax.xml.xpath` + StAX
html	Sélecteur CSS, `@attr` pour un attribut	`td.nom`, `a@href`	Jsoup
csv / xlsx	Nom ou index de colonne	`Nom`, `#3`	Commons CSV, POI

4. Règles de valeur

Chaque propriété FtM reçoit une règle qui produit une **liste de chaînes** (FtM est multi-valué). Une règle a exactement **une source** (§ 4.1), puis des étapes optionnelles appliquées dans un ordre fixe (§ 4.2).

4.1 Sources de valeur (une seule par règle)

Clé	Produit	Exemple
`path`	Toutes les valeurs trouvées par le sélecteur	`{ path: "$.pays[*]" }`
`paths` + `join`	Une valeur par nœud : les chemins concaténés avec `join` (défaut `" "`), les vides ignorés	`{ paths: ["$.prenom", "$.nom"], join: " " }`
`template`	Une chaîne avec `{sélecteur}` remplacés	`{ template: "https://x.org/notice/{$.id}" }`
`literal`	Une ou plusieurs constantes	`{ literal: "crime" }`, `{ literal: ["lu", "eu"] }`
`ref`	L'ID d'une autre entité, recalculé avec son `ns` + `keys` (§ 6)	`{ ref: { ns: person, keys: ["@ProfileID"] } }`

Forme courte : une chaîne seule vaut `{ path: "…" }`. `name: "$.fullName"` équivaut à `name: { path: "$.fullName" }`.

4.2 Étapes optionnelles, dans cet ordre

1. `where` : filtre les **nœuds** trouvés par `path` avant d'en lire le texte (ex. ne garder que les `NamePart` dont `@TypeID` vaut « First Name » dans un lookup).
2. `field` : lit un sous-sélecteur dans chaque nœud trouvé (ou dans l'élément retourné par un lookup).
3. `lookup` / `chain` : remplace chaque valeur par sa valeur dans une table ; `chain` enchaîne plusieurs lookups.
4. `split` : découpe chaque valeur sur un séparateur (ex. `";"`).
5. `transform` : un transformer ou une liste de transformers appliqués en séquence (§ 5).
6. `default` : valeur utilisée si la liste est vide à ce stade.
7. `required: true` : liste encore vide → le fragment est rejeté et logué.

Après la dernière étape, le moteur retire les valeurs vides et les doublons avant `addValue`.

4.3 Lookup dans une règle

```
address:
  path: "Feature[@Type='Address']/@LocationID"
  lookup: locations          # élément Location entier
  field: "City"              # puis on lit City dedans
country:
  path: "Feature[@Type='Address']/@LocationID"
  chain:
    - { lookup: locations, field: "CountryRef/@ID" }
    - { lookup: countries }  # 11 → IR
  transform: country          # IR → ir
```

Code absent de la table : la valeur est retirée et un avertissement `lookup_miss` est compté (§ 8). `on_miss: keep` garde la valeur brute.

4.4 Conditions (`where` et `when`)

Forme	Vrai si
`{ attr: "@Type", equals: "Birthdate" }`	Le sélecteur vaut la constante
`{ attr: "@TypeID", lookup: name_part_types, equals: "First Name" }`	La valeur traduite par le lookup vaut la constante
`{ attr: "$.status", in: ["active", "current"] }`	La valeur est dans la liste
`{ attr: "$.endDate", exists: false }`	Le sélecteur ne trouve rien
`{ attr: "td.role", matches: "(?i)ancien" }`	Regex Java
`{ all: […] }` , `{ any: […] }` , `{ not: {…} }`	Combinaisons

`where` filtre des nœuds dans une règle ; `when` filtre le nœud racine d'un fragment entier.

4.5 Exemple de propriété complète

```
firstName:
  path: "NamePart"
  where: { attr: "@TypeID", lookup: name_part_types, equals: "First Name" }
  transform: [trim, collapse_spaces]
  required: true
```

4.6 Schéma calculé

```
schema:
  path: "@PartyTypeID"
  lookup: party_types
  map: { Individual: Person, Entity: Organization, Vessel: Vessel }
  default: LegalEntity
```

Valeur hors `map` sans `default` : fragment rejeté (`schema_unknown`).

5. Catalogue des transformers

Un transformer prend **une chaîne** et rend **zéro, une ou plusieurs chaînes**. Rendre zéro valeur = la valeur est écartée (et comptée en `transform_drop`). Un transformer ne lève jamais d'exception vers le moteur.

5.1 Syntaxe

```
birthDate: { path: "$.dob", transform: date }                          # nom
seul
birthDate: { path: "$.dob", transform: { date: { formats: ["dd.MM.yyyy"] } }
}   # avec paramètres
name:      { path: "$.n",   transform: [trim, collapse_spaces, { case: title
}] }  # chaîne, appliquée de gauche à droite
```

5.2 Texte

Nom	Paramètres	Entrée → sortie
`trim`	—	`" Jean "` → `"Jean"`
`collapse_spaces`	—	`"Jean  Dupont"` → `"Jean Dupont"` (espaces insécables inclus)
`case`	`upper` | `lower` | `title`	`"DUPONT"` + `title` → `"Dupont"`
`strip_html`	—	`"<b>Jean</b>"` → `"Jean"`
`regex_extract`	`pattern` , `group` (défaut 1)	`"Né le 12.03.1970 à Esch"` + `pattern: "(\\d{2}\\.\\d{2}\\.\\d{4})"` → `"12.03.1970"` ; pas de match → rien
`regex_replace`	`pattern` , `replacement`	`"M. Jean"` + `pattern: "^(M\.`

Nom	Paramètres	Entrée → sortie
`split`	`sep` (chaîne ou regex)	`"FR; BE"` + `sep: ";"` → `"FR"`, `"BE"`
`prefix` / `suffix`	`value`	`"123"` + `prefix: "https://x.org/"` → `"https://x.org/123"`
`map`	table inline, `default`	`"M"` + `{ M: male, F: female }` → `"male"`

5.3 Noms

Nom	Paramètres	Entrée → sortie
`name`	—	`trim` + `collapse_spaces` + retrait des titres (`M.`, `Dr`, `S.E.`) : `"Dr Jean DUPONT"` → `"Jean DUPONT"`
`name_swap`	`sep` (défaut `","`)	`"DUPONT, Jean"` → `"Jean DUPONT"`

5.4 Dates

Nom	Paramètres	Entrée → sortie
`date`	`formats` (liste de motifs `DateTimeFormatter`, essayés dans l'ordre), `locale` (défaut `fr`)	`"12.03.1970"` → `"1970-03-12"` ; `"mars 1970"` → `"1970-03"` ; `"1970"` → `"1970"`
`date_range`	`sep` (défaut `" - "`), `part` : `start` | `end`	`"2018 - 2023"` + `part: end` → `"2023"`

La précision est conservée : FtM accepte `yyyy`, `yyyy-MM` et `yyyy-MM-dd`. Le moteur n'invente jamais un jour ou un mois absent. Format inconnu → valeur écartée + `transform_drop`.

5.5 Codes et identifiants

Nom	Paramètres	Entrée → sortie
`country`	—	Code ISO-2 FtM en minuscules : `"FR"`, `"FRA"`, `"France"`, `"Luxembourg"` → `"fr"`, `"lu"` ; non reconnu → écarté
`gender`	—	`"M"`, `"Male"`, `"Homme"` → `"male"` ; `"F"`, `"Femme"` → `"female"`

Nom	Paramètres	Entrée → sortie
`email`	—	Minuscules + contrôle de forme ; invalide → écarté
`url`	`base` (pour les liens relatifs)	`"/fr/depute/17"` + `base: "https://chd.lu"` → `"https://chd.lu/fr/depute/17"`
`phone`	`region` (défaut `LU`)	Format E.164 : `"621 123 456"` → `"+352621123456"`

5.6 Transformers personnalisés

Un nom absent du catalogue est cherché dans le registre (§ 7). Un plugin PF4J peut y ajouter ses propres transformers, par exemple `ofac_date_period`, puis les citer dans son mapping comme les autres.

Règle de conception : dès qu'une transformation demande un `if`, une boucle ou un état, elle devient un transformer Java nommé. Le langage du YAML n'est jamais étendu pour ça.

6. IDs, fusion et validation

6.1 ID déterministe

```
id: { ns: person, keys: ["@ProfileID"] }
```

Formule :

```
\mathrm{id} = \mathrm{dataset} + \text{"-"} + \mathrm{hex}\big(\mathrm{SHA1}
(\mathrm{dataset} \,|\, \mathrm{ns} \,|\, k_1 \,|\, k_2 \,|\, \dots)\big)
```

Exemple : `dataset = ofac`, `ns = person`, `@ProfileID = 36` → `ofac-` + sha1(`ofac|person|36`).

- Les valeurs de `keys` passent par `trim` avant le hash. Une clé vide rejette le fragment (`id_missing`).
- `ns` est obligatoire : il empêche qu'une `Person` 36 et une `Sanction` 36 aient le même ID.
- Une constante s'écrit entre apostrophes : `keys: ["@ID", "'red-notice'"]`.
- `ref` utilise exactement la même fonction. Une `Sanction` pointe donc vers une `Person` **sans l'avoir lue**, en recalculant son ID.

6.2 Fusion des fragments

Plusieurs fragments peuvent produire le même ID (ex. `SanctionData` et `Link` d'une même sanction). Avant l'écriture, le moteur les fusionne :

1. **Valeurs :** union par propriété, sans doublon, ordre d'arrivée conservé.
2. **Schéma :** identique, ou le plus spécifique si l'un étend l'autre (`LegalEntity` + `Person` → `Person`). Deux schémas sans lien (`Person` + `Vessel`) → conflit `schema_conflict`, les deux fragments sont rejetés.
3. **Métadonnées :** `datasets` réunis.

Volume	Stratégie
Jusqu'à ~1 million d'entités	`HashMap<String, ValueEntity>` en mémoire
Au-delà	Fragments écrits en NDJSON temporaire, tri externe par ID, fusion des lignes consécutives de même ID

6.3 Validation finale

Après fusion, chaque entité est contrôlée contre le modèle FtM (`Model.loadDefault()`) :

- **Propriétés obligatoires du schéma présentes.** Par exemple, une `Sanction` sans `entity` est rejetée (`required_missing`).
- **Valeurs conformes au type de la propriété.** Date, pays, entité : une valeur invalide est retirée (`type_invalid`).
- **Références résolues.** Un `ref` vers un ID absent du lot donne un avertissement `dangling_ref` , mais l'entité est gardée.
- **Une entité sans aucune propriété est rejetée.**

6.4 Validation du mapping au démarrage

Avant de lire le moindre fichier, le moteur vérifie :

- **Les noms.** Chaque `schema` et chaque propriété existent dans le modèle FtM.
- **Les références internes.** Chaque `lookup` , `ns` de `ref` et `transform` cité est défini.
- **La syntaxe.** Les sélecteurs compilent dans le langage du `format` .

La première erreur arrête tout, avec le chemin dans le YAML (ex. `fragments[1].properties.birthdate: propriété inconnue pour Person`).

7. Contrats Java

Quatre interfaces suffisent. Seuls `RecordReader` (un par format) et `Transformer` (un par transformation) ont plusieurs implémentations ; le reste est écrit une fois.

```
/** Un par format : json, xml, html, csv, xlsx. Seul code dépendant du
format. */
public interface RecordReader extends AutoCloseable {
    String format();
    void open(InputStream in, ReaderOptions options) throws IOException;
    /** Nœuds correspondant au sélecteur, lus en streaming quand le format le
permet. */
    Stream<Node> select(String selector);
}

/** Un nœud opaque (JsonNode, org.w3c.dom.Node, Jsoup Element, ligne CSV). */
public interface Node {
    List<Node> nodes(String selector);   // sous-nœuds relatifs
    List<String> values(String selector); // textes ou attributs relatifs
    String text();
}

/** Une transformation nommée. Sans état, thread-safe. */
public interface Transformer {
    String name();
    List<String> apply(String value, Map<String, Object> params);
    /** Appelé au démarrage : refuser ici les paramètres invalides. */
    default void validate(Map<String, Object> params) {}
}

/** Le moteur : mapping + fichier → entités FtM fusionnées. */
public interface MappingEngine {
    MappingConfig load(Path mappingFile);                // YAML ou JSON,
validé (§ 6.4)
    RunReport run(MappingConfig config, Path source, Path output);
}
```

7.1 Registres

Registre	Clé	Remplissage
`ReaderRegistry`	`format`	Beans Spring `RecordReader`
`TransformerRegistry`	`name`	Beans Spring `Transformer` + extensions PF4J (`@Extension implements Transformer`)

Un nom présent deux fois fait échouer le démarrage, pour ne jamais masquer un transformer en silence.

7.2 Modèle de configuration

`MappingConfig`, `LookupDef`, `FragmentDef`, `ValueRule`, `Condition` et `IdDef` sont des `record` Java immuables. Ils sont lus avec Jackson : `YAMLFactory` pour `.yaml`, `JsonFactory` pour `.json`. YAML et JSON ont donc **exactement** le même schéma, et un JSON Schema publié (`mapping.schema.json`) permet l'autocomplétion dans l'IDE.

7.3 Place dans swift-check-aml

- Le moteur vit dans `swiftcheck-aml-core` et les mappings dans un dossier `mappings/`, rechargés sans redémarrage.
- Un plugin PF4J n'est plus nécessaire pour une source « normale ». Il reste l'échappatoire pour les sources complexes (PDF, HTML déstructuré) via `SourceParser`, ou pour fournir des transformers spécifiques.

8. Erreurs et rapport d'exécution

Principe : **une erreur de mapping arrête tout, une erreur de donnée n'arrête rien.** Une mauvaise ligne est rejetée, comptée et loguée, et le reste du fichier continue.

Code	Niveau	Effet
`config_invalid`	fatal	Arrêt avant lecture (§ 6.4)
`source_unreadable`	fatal	Fichier absent, encodage ou XML invalide
`id_missing`	rejet	Clé d'ID vide : fragment ignoré
`schema_unknown`	rejet	Schéma calculé hors `map`
`schema_conflict`	rejet	Fusion de deux schémas incompatibles
`required_missing`	rejet	Propriété `required` ou obligatoire FtM vide

Code	Niveau	Effet
`lookup_miss`	avertissement	Code absent d'une table
`transform_drop`	avertissement	Valeur écartée par un transformer
`type_invalid`	avertissement	Valeur refusée par le type FtM
`dangling_ref`	avertissement	`ref` vers un ID absent du lot

Chaque log porte : dataset, nom du fragment, position dans la source (index de nœud ou ligne), propriété, valeur brute.

8.1 Rapport

Chaque exécution écrit `<dataset>.report.json` à côté de la sortie :

```
{
  "dataset": "ofac",
  "mappingVersion": 1,
  "source": "sdn_advanced.xml",
  "sourceSha256": "9f2c…",
  "startedAt": "2026-10-22T21:00:00Z",
  "durationMs": 41250,
  "nodesRead": 18342,
  "fragments": 25110,
  "entities": { "Person": 6120, "Organization": 4011, "Sanction": 10131 },
  "rejected": { "id_missing": 3, "required_missing": 12 },
  "warnings": { "lookup_miss": 41, "transform_drop": 208 }
}
```

8.2 Seuil de qualité

`quality.max_rejected_ratio` (ex. `0.02`, clé racine optionnelle) : au-delà de 2 % de fragments rejetés, l'exécution est marquée en échec et la sortie précédente est conservée. Un changement de structure du site source ne remplace ainsi jamais une bonne liste par une liste vide.

9. Exemples complets

9.1 JSON simple : Interpol (YAML)

```
version: 1
dataset: interpol
format: json
defaults:
  topics: { literal: "crime" }

fragments:
  - name: person
    node: "$.notices[*]"
    schema: Person
    id: { ns: person, keys: ["$.id"] }
    properties:
      firstName:   { path: "$.prenom", transform: name }
      lastName:    { path: "$.nom", transform: name }
      name:        { paths: ["$.prenom", "$.nom"], join: " ", required: true
}
      birthDate:   { path: "$.naissance", transform: { date: { formats:
["yyyy/MM/dd"] } } }
      nationality: { path: "$.pays[*]", transform: country }
      sourceUrl:   { template: "https://www.interpol.int/notice/{$.id}" }
```

9.2 Le même mapping en JSON

```
{
  "version": 1,
  "dataset": "interpol",
  "format": "json",
  "defaults": { "topics": { "literal": "crime" } },
  "fragments": [
    {
      "name": "person",
      "node": "$.notices[*]",
      "schema": "Person",
      "id": { "ns": "person", "keys": ["$.id"] },
      "properties": {
        "firstName":   { "path": "$.prenom", "transform": "name" },
        "lastName":    { "path": "$.nom", "transform": "name" },
        "name":        { "paths": ["$.prenom", "$.nom"], "join": " ",
"required": true },
        "birthDate":   { "path": "$.naissance", "transform": { "date": {
"formats": ["yyyy/MM/dd"] } } },
```

```
        "nationality": { "path": "$.pays[*]", "transform": "country" },
        "sourceUrl":   { "template": "https://www.interpol.int/notice/{$.id}"
}
      }
    }
  ]
}
```

9.3 HTML : députés de la Chambre (PEP Luxembourg)

Un député donne trois entités : la `Person`, la `Position` (une seule pour tous) et l'`Occupancy` qui les relie.

```
version: 1
dataset: lu_chamber
format: html

fragments:
  - name: person
    node: "table.deputes tr:has(td)"
    schema: Person
    id: { ns: person, keys: ["td.id"] }
    properties:
      name:      { path: "td.nom", transform: [name, name_swap] }
      birthDate: { path: "td.ne", transform: { date: { formats:
["dd.MM.yyyy"] } } }
      country:   { literal: "lu" }
      topics:    { literal: "role.pep" }
      sourceUrl: { path: "td.nom a@href", transform: { url: { base:
"https://www.chd.lu" } } }

  - name: position
    node: "table.deputes"                  # un seul nœud → une seule
Position
    schema: Position
    id: { ns: position, keys: ["'member-of-parliament'"] }
    properties:
      name:    { literal: "Membre de la Chambre des Députés" }
      country: { literal: "lu" }
      topics:  { literal: "gov.national" }

  - name: occupancy
    node: "table.deputes tr:has(td)"
    schema: Occupancy
    id: { ns: occupancy, keys: ["td.id"] }
    properties:
```

```
      holder:    { ref: { ns: person, keys: ["td.id"] } }
      post:      { ref: { ns: position, keys: ["'member-of-parliament'"] } }
      startDate: { path: "td.debut", transform: date }
      status:    { literal: "current" }
```

9.4 XML relationnel : OFAC Advanced (extrait)

Les tables de codes passent par `lookups`. Les liens Person ↔ Sanction passent par `ref` et la fusion.

```
version: 1
dataset: ofac
format: xml

lookups:
  party_types:     { path: "//PartyType",    key: "@ID", value: "text()" }
  name_part_types: { path: "//NamePartType", key: "@ID", value: "text()" }
  countries:       { path: "//Country",      key: "@ISO2" }
  locations:       { path: "//Location",     key: "@ID" }

fragments:
  - name: party
    node: "//Profile"
    schema:
      path: "@PartyTypeID"
      lookup: party_types
      map: { Individual: Person, Entity: Organization }
    id: { ns: party, keys: ["@ID"] }
    properties:
      firstName:
        path: "NamePart"
        where: { attr: "@TypeID", lookup: name_part_types, equals: "First
Name" }
        transform: name
      lastName:
        path: "NamePart"
        where: { attr: "@TypeID", lookup: name_part_types, equals: "Last
Name" }
        transform: name
      name:
        paths: ["NamePart[@TypeID='1521']", "NamePart[@TypeID='1520']"]
        join: " "
        required: true
      birthDate: { path: "Feature[@Type='Birthdate']", transform: date }
      address:
        path: "Feature[@Type='Address']/@LocationID"
```

```
        lookup: locations
        field: "City"
      country:
        path: "Feature[@Type='Address']/@LocationID"
        chain:
          - { lookup: locations, field: "CountryRef/@ID" }
          - { lookup: countries }
        transform: country
      topics: { literal: "sanction" }

  - name: sanction
    node: "//SanctionsEntry"
    schema: Sanction
    id: { ns: sanction, keys: ["@ID"] }
    properties:
      entity:    { ref: { ns: party, keys: ["@ProfileID"] }, required: true }
      program:   { path: "Program" }
      startDate: { path: "EntryDate", transform: date }
      authority: { literal: "OFAC" }
```

Sortie attendue pour le profil 36 : une `Person` `ofac-<sha1(ofac|party|36)>` et une `Sanction` dont `entity` vaut ce même ID, écrites chacune sur une ligne de `ofac.ftm.json`.